Three ways in, 720 to the team fast, full res behind it

19 Oct 2026 · architecture proposal, nothing changed yet

The shape asked for:

Route A	a script auto-selects clips from drone footage into the upload watch folder
Route B	local import of a day's raw footage (20 GB+), reviewed on the laptop, **selected** media uploaded
Route C	direct upload of media — videos *and* photos — no review step
Viewing	one way to watch **high res**: the MacBook, on a monitor, from the **original**
Evening	full-res goes to the cloud overnight; the ladder **extends** to full res then
Goal	selected clips to the team at **720, as soon as possible**

---

1. First, a retraction: "two ladders per clip" is the design, not a bug

I flagged U3 in the previous document — every clip produces **two** Bunny Stream videos, one from the proxy and one from the original, two adaptive ladders, two sets of renditions — and ranked "one ladder per clip" third on the fix list.

That fix would have broken exactly what you have just asked for. 720 now and full res later *is* two ladders. The machinery is already there: `putAndFetch(kind: 'proxy' | 'original')` builds one from each (`video-rendition-sync.ts:142`), and `url/route.ts` already prefers the original's ladder when it exists and falls back to the proxy's (`resolveOriginal` / `resolveProxy`, `:125`–`:145`). Playback for a two-stage clip works today.

What is actually wrong is **when** and **from what**:

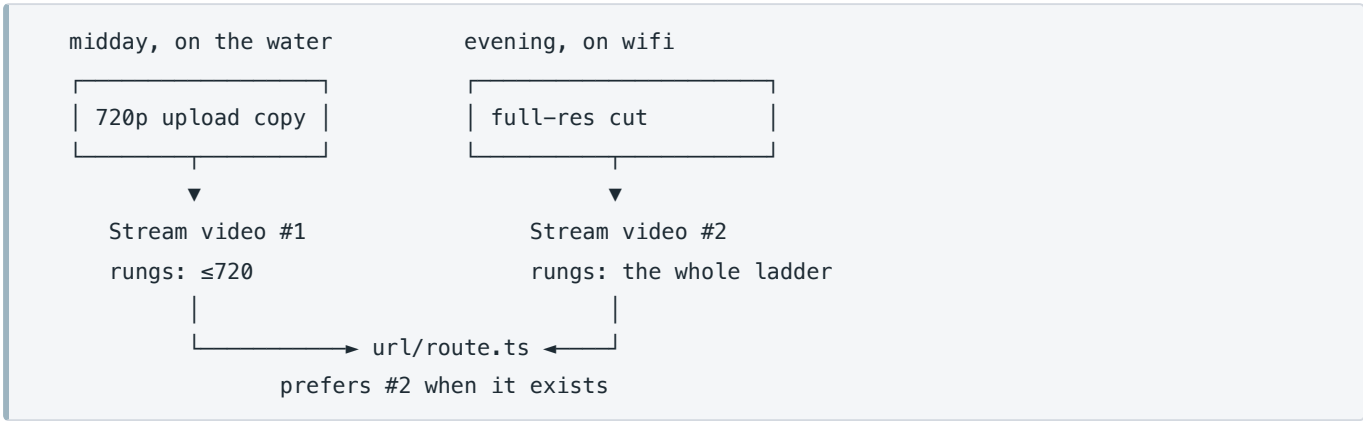
	Today	**Should be**
Fast ladder source	1.2 Mbps proxy — a 5× cut of a 6.1 Mbps 720p clip	~the 720 rung's own bitrate, so the rung is not a re-encode of a starved file
Full ladder source	the `~/clips` 720p file, labelled "original"	the **actual full-res cut**
Timing	both queued in the same session, competing for one uplink	720 now; full res on a schedule

So the correction is: keep both ladders, fix their sources and sequence them. Not a rewrite — a re-pointing.

---

2. The destination contract

One Bunny Stream library, the **full ladder enabled** (360 / 480 / 720 / 1080 / 2160 as wanted). Two uploads per clip, at different times:



This rests on one assumption: Bunny Stream does not upscale. A 720p source in a library with 1080p and 4K enabled should produce rungs only up to 720. That is standard ABR behaviour and almost certainly what Bunny does — but **their documentation does not say so**, and I could not confirm it. See §7; it is a five-minute test and the whole design depends on it.

If Stream *does* upscale, the fallback is two libraries (one ≤720, one full) and route the two uploads accordingly. Workable, slightly more config, same shape.

Why this avoids the "configure before upload" trap

Bunny's encoding docs are explicit that "*the resolutions need to be configured before a video is uploaded*" and "*videos only uploaded after the changes will be affected*". So you cannot upload a clip under a ≤720 ladder and later extend **that video's** ladder — not without Keep-Originals and a re-encode, and the kept original would be the 720p proxy, so the extra rungs would be upscaled rubbish.

Two Stream videos sidesteps it entirely. Nothing is ever reconfigured; the second upload simply brings a better source and a fuller ladder with it.

Set the 720 rung's bitrate deliberately

Per-resolution bitrate **is** configurable in the library (click the number beside the resolution). Bunny publish no default figure. Whatever it is set to is the ceiling on what the team actually sees, and it is also the number the upload copy should be encoded at — anything above it is uploaded and then thrown away by the transcode.

Measured on 2 Oct: the `~/clips` files average **6.10 Mbps** at 720p, four of fifteen pinned against the `--maxrate 8M` cap. If Stream's 720 rung is ~3 Mbps, roughly half of today's fast-path bytes are being uploaded to be discarded.

Fast-path source	MB for 22 min	relative
today's `~/clips` file, 6.10 Mbps	1027	1.0×
3 Mbps 720p upload copy	505	**0.49×**
today's proxy, 1.2 Mbps	202	0.20×

And the crossover is comfortable: transcoding to 3 Mbps before sending beats sending 6.1 Mbps as-is on any uplink **below ~23 Mbps**, assuming a hardware transcode at ~6× realtime. Marina and hotel links are well below that, so the transcode is free in wall-clock terms and halves the bytes.

---

3. Route A — the script into the watch folder

Exists, end to end. `clips:day` cuts on SSA's own tags, writes to `~/clips`, Upload tab → *Watch clips* picks them up as the encoder finishes each one.

Two changes:

1. **Make `clips:day`'s output the upload copy.** It is already 720p; encode it at the 720 rung's bitrate rather than CRF 26 capped at 8M. One flag change, and SSA then uploads it **without transcoding again** — which is the proxy step disappearing on this route, not because the proxy is pointless but because the script has already made it.
2. **Keep the manifest for the evening.** `--full-res <manifest>` already re-cuts the same segments at source resolution (`select-race-clips.mjs:299`), and `--copy` cuts without re-encoding — "*lossless and near instant*". So the evening's full-res cuts are a replay of a file that already exists, not a second selection pass.

Net: the day's fast upload halves, and the archive cut becomes a one-liner.

Route B — local review of 20 GB+, then upload the selection

Half exists. `FootageReview.tsx` already points at `<YYYYMMDD>/Drone` on the SSD via a persisted `FileSystemDirectoryHandle` (`prefsStore.js:20`) and plays the camera's **own** `.LRF` proxies — 720p, ~7–8 Mbps, "*roughly a thirtieth of the bytes, already on the SSD*" (`droneProxy.ts:5`). Nothing is copied. **G** drops a Grab tag on the frame being watched.

What is missing is the step from "I marked this" to "it is uploading".

The one rule that matters here: the 20 GB must not enter IndexedDB

`saveVideo` puts the `File` itself into IndexedDB (`localStore.js:317`), so a 20 GB import is a 20 GB copy from the SSD onto the internal disk — and then every `getAllVideos()` deserialises all of it, three times at boot (`SSAApp.jsx:531,535,543`).

That is the finding I could not confirm last time because the test browser held **0 clips**. Under this route it stops being hypothetical: **a 20 GB local library is precisely the condition that makes it bite.** So the rule is to keep the bytes where they are:

- review plays the `.LRF` off the handle, as it does now;
- a selection records a **path + in/out seconds**, not a blob;
- on upload, read that window from the original through the handle, transcode just that window to the 720 upload copy, send it, and keep nothing.

The library then holds metadata and selections — kilobytes — and the IndexedDB cost never arises. It also means route B needs no `DB_VER` change.

Route C — direct upload, videos and photos

Exists, and is the slowest per item. No review step, so this is the phone or the laptop sending straight up. The work here is the per-object overhead from the previous document, unchanged:

- photo sync is serial across every date and photo, ~6 round trips each (`photoStore.js:527`) — 100 photos is ~10 minutes of pure latency at 80 ms;
- `photoStore.getCreds()` is **uncached** and called twice per photo (`:432,446,463`) — and I made that route do a Supabase auth plus a `users` query last month, so those two calls are now the expensive kind;
- `auth.getUser()` sits **inside** the originals loop (`useCloudSync.js:245`);
- each photo is decoded at full resolution **twice** on the main thread, and `createImageBitmap` appears nowhere in the codebase — hence "*≈2 min each*" for the 170 MP compilations, in our own docs.

None of that is architectural. It is a credentials cache, a hoisted call, a pool of 4–6, and one decode at target size in a Worker.

---

4. Viewing high res on the monitor — INSIDE SSA, with the overlays

The overlay is the whole point, so "open it in QuickTime" is not an answer. I withdraw that suggestion from the previous version: it loses the instruments, which is the reason to watch it at all.

The good news: this is already built

`VideoPlayer.jsx` has a **"◆ HD local"** toggle, gated to coach/admin (`canPlayLocalHD`, `SSAApp.jsx:1524`). Pressing it does exactly this (`VideoPlayer.jsx:271`):

```
if (useLocalHD && video.hasLocalBlob) {
  const blob = await getVideoBlob(video.id)
  createdBlobUrl = URL.createObjectURL(blob)
  srcUrl = createdBlobUrl               // progressive, no streaming
}
```

…and the gauges carry on drawing over it untouched, because the overlay is driven by `logData` + `interpRow(logData.rows, logUtc)` (`:387`) and does not care where the pixels came from. **Requirement (ii) ships today.** It is labelled "*Play the original from local storage (HD, no streaming)*" and is described in its own comment as the path "*used for debriefs*".

The one thing in the way

It is gated on `video.hasLocalBlob` — the bytes must be **in IndexedDB**, which is the 20 GB copy route B is trying to avoid.

So the change is small and well-shaped: give the local-source resolver a second place to look.

```
resolveLocalSource(video) →
    1. the persisted folder handle   (new — a File, read on demand, no copy)
    2. IndexedDB getVideoBlob(id)    (today's path, unchanged)
    3. null → the button stays hidden, as now
```

One function, one extra branch. It decouples high-res viewing from the IndexedDB copy, which is what lets route B keep 20 GB on the SSD and still offer the overlay at full resolution.

The real constraint: can the browser decode the file?

This is the part to settle first, because it is the only thing that can make in-SSA high-res viewing impossible.

`showDirectoryPicker`	**Chromium only.** Not in any released Safari (caniuse lists "not supported" through 27.2) — which is why `FootageReview.tsx:224` already says "*Chrome/Edge only — Safari has no folder picker*"
HEVC / H.265 playback	Safari 13+ **full**. Chrome **"partial" from 107** — in practice hardware-decoded 8-bit 4:2:0 on Apple silicon, and typically **not 10-bit**

So the two capabilities sit in different browsers, and whether that collides depends entirely on what the drone records:

- **H.264** → Chrome plays it. Nothing to do; the handle branch above is the whole job.
- **HEVC 8-bit 4:2:0** → Chrome on an Apple-silicon Mac should play it. Test it.
- **HEVC 10-bit** (D-Log, HLG) → Chrome will almost certainly refuse, and this is the likely setting for anyone shooting to grade. `videoProbe.js:238` already carries the error message for it, which suggests it has happened.

Settle it with one command, with the drive connected:

```
ffprobe -v error -select_streams v:0 -show_entries
stream=codec_name,profile,pix_fmt,width,height,bit_rate -of default=nw=1
"/Volumes/JonJon1/DCIM/DJI_001/DJI_20261002120310_0023_D.MP4"
```

`codec_name=h264` means there is no problem at all. `hevc` with `pix_fmt=yuv420p` is probably fine in Chrome. `yuv420p10le` means it is not.

And if the answer is the bad one, it still works

Decodability is only required for the file SSA plays — and we choose that file. The camera original never has to be browser-decodable; for the evening archive it is only bytes to be uploaded, and nothing decodes it.

So on an HEVC-10-bit card, `clips:day` produces **two** local renditions instead of one:

	For	Lives
720p at the ladder's bitrate	the team, uploaded immediately	`~/clips`, then Bunny
1080p H.264 viewing copy	high res in SSA, with overlays	the MacBook only

`compress-videos.sh` already does hardware H.264 through videotoolbox (`:41`) with a configurable scale and cap, so this is a second invocation, not new code. At 1080p/12 Mbps a 22-minute day is roughly 2 GB of local disk — and on a 27-inch monitor 1080p at a generous bitrate is a large step up from 720 without the cost of 4K.

That makes the high-res path independent of the camera's codec choice, which is worth having regardless of how the `ffprobe` comes out.

5. The evening full-res upload

A scheduled, unattended drain. The pieces exist — `enqueueOriginals`, `goodForOriginals` honouring Save-Data and the Wi-Fi-only toggle, and `processOriginalsQueue` already yielding to the proxy queue (`useCloudSync.js:230`: "*let proxies finish first*"). What is missing is that it runs whenever the link looks good, which during the day means competing with the very uploads that are meant to be fast.

Three changes:

1. **Gate it on a window, not just link quality** — after a set hour, or an explicit "upload originals now" the user leaves running.
2. **Send the full-res cut, not the `~/clips` file.** Today the originals queue uploads what route A already downsized; under this design it uploads the `--full-res` output.
3. **Point it at storage-first.** The background queue still uses `uploadFileToStream` (TUS → Stream, `useCloudSync.js:291`) while the batch button uses `uploadOriginalStorageFirst` (`useBatchActions.js:260`). The TUS path leaves a clip unwatchable for 60–120 min. It matters less for an evening archive than it did for the fast path, but there is no reason to keep two implementations and the slow one on the automatic route.

---

6. What this leaves of the duplicate problem

Content addressing (U2/U7 previously) still stands, and this architecture makes it **more** valuable, not less: each clip now has two cloud objects and two Stream videos, and the only thing tying the pair together is a local row.

The fingerprint is still nearly free — `extractVideoCreationTime` already reads 1 MB of head and 2 MB of tail for every clip (`videoProbe.js:283`), so `SHA-256(size ‖ head ‖ tail)` costs a digest over bytes already in memory. With it:

- the 720 copy and the full-res cut of the same source clip are **provably** the same clip, rather than two rows that happen to share a name;
- a second laptop importing the same card computes the same key and cannot upload a duplicate — today `videos` dedupes on `external_id`, which is the local `v_<timestamp>_<random>` id (`localStore.js:270`), so it is per-device by construction;
- `storageObjectSize`'s per-clip directory listing (`bunny-storage-upload.ts:170`) can be replaced by one cached listing per prefix — 20 listings of the same directory becomes 1, which also stays clear of Bunny's documented ceiling of 5 concurrent folder listings per zone.

930 videos are in the library today. If that is two per clip, the pairing is already carrying more weight than a random local id should be asked to bear.

---

7. Two things to verify before building any of it

1. Does Stream skip rungs above the source resolution? The whole two-upload design assumes yes. Upload one 720p test clip to the library and look at the renditions it produces. If 1080p and 4K rungs appear, the design needs two libraries instead of one.

2. What is the library actually set to? I could not read it: the `BUNNY_STREAM_API_KEY` in `.env.local` is the **library** key, and `GET /library/620505` with it returns only counts — `EnabledResolutions`, `Bitrate720p`, `KeepOriginalFiles` and `AllowEarlyPlay` need the **account** API key, which is not in the project's environment. The dashboard answers it in thirty seconds: Stream → library → Encoding.

The numbers worth having from that screen: which resolutions are on, and what the **720p bitrate** is set to — because that is the figure `clips:day` should encode to, and the one that decides whether the fast path halves or not.

Also worth a look while you are there: `AllowEarlyPlay`. Early-Play lets a clip be watched *before* the ladder finishes, by serving the source file directly — which is the "ASAP" goal, for free. The catch is that it "*exposes the original file publicly*", and per the Bunny playback findings this library runs with Block-Direct-URL on and token auth precisely to stop that. On the fast path the exposed file would be the 720p upload copy rather than the archive, so it may be an acceptable trade — but it is a security decision, not a performance one, and it is yours.

---

8. Order

#	Change	Why now
1	Read the library's Encoding screen; test the no-upscale assumption (§7); `ffprobe` one drone original (§4)	everything else is shaped by these three answers, and all three are yours to run
2	`clips:day` encodes the upload copy at the 720 rung's bitrate; SSA stops re-transcoding it	halves route A's fast-path bytes, removes a transcode per clip
3	Cache creds, hoist `getUser()`, pool the photo sync, one decode per photo	route C's whole cost; no architecture involved
4	One cached prefix listing instead of one per clip	removes the quadratic existence check and the 5-listing ceiling
5	Evening queue: windowed, fed the `--full-res` cut, on storage-first	makes (iii) real and gets the slow path off the fast path's uplink
6	`resolveLocalSource`: let "◆ HD local" read from the folder handle, not only IndexedDB	**requirement (ii), inside SSA with overlays** — one extra branch on a feature that already ships
7	Route B: selection → window transcode → upload, **without** IndexedDB	the 20 GB route; shares the handle with item 6

#	Change	Why now
8	Content fingerprint + `content_hash` + content-addressed keys	pairs the two uploads properly and makes cross-device duplicates impossible

1 is half an hour of yours and nothing of mine. 2–4 are mechanical and independently shippable. **6 is small and gets (ii) working off the folder instead of a 20 GB copy** — I would do it early, before route B, because it is useful on its own. 7 is the real build.

I have been wrong twice in this document's lineage by estimating a number I could have measured — the clip bitrate, and then the empty local library. Item 1 is there so it does not happen a third time.